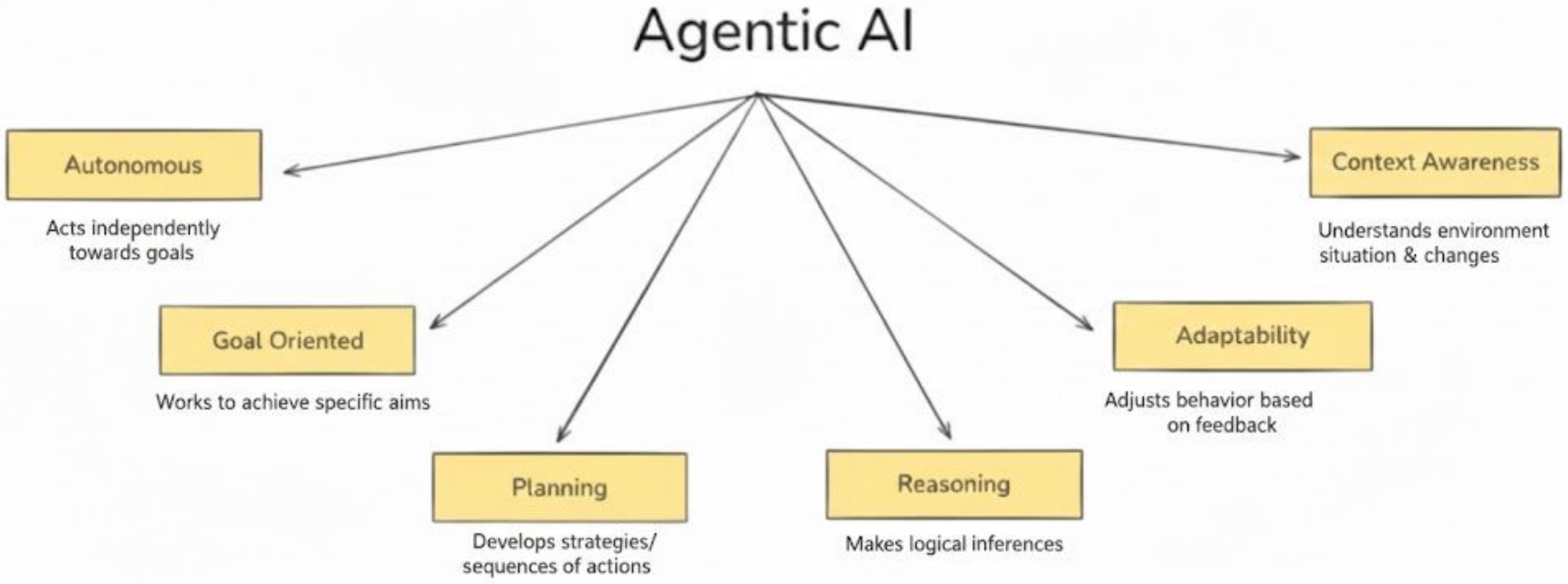
Lang-graph

What is Agentic AI

- An Agentic AI is a type of AI that can take up a task or goal from an user and then work towards completing it with minimal human guidance . It plans, takes action, adapts to changes, and seeks help only when necessary.

What is Agentic AI



What is Agentic AI

- **Autonomy:**
- Autonomy refers to the AI system's ability to make decisions and take actions on its own to achieve a given goal, without needing step-by-step human instructions.
- Autonomy in multiple facets
  - a. Execution
  - b. Decision making
  - c. Tool usage

What is Agentic AI

- **Autonomy:**
- Autonomy can be controlled
  - a. Permission Scope - Limit what tools or actions the agent can perform independently. (Can screen candidates, but needs approval before rejecting anyone.)
  - b. Human-in-the-Loop (HITL) - Insert checkpoints where human approval is required before continuing. (Can I post this JD?)
  - c. Override Controls - Allow users to stop, pause, or change the agent's behaviour at any time. (Pause screening command to halt resume processing.)
  - d. Guardrails / Policies - Define hard rules or ethical boundaries the agent must follow. (Never schedule interviews on weekends)

What is Agentic AI

- **Goal Oriented:**
- Being goal-oriented means that the AI system operates with a persistent objective in mind and continuously directs its actions to achieve that objective, rather than just responding to isolated prompts.
  - Goals acts as a compass for Autonomy
  - Goals can come with constraints
  - Goals are stored in core memory
  - Goals can be altered

What is Agentic AI

- **Planning:**
- Planning is the agent's ability to break down a high-level goal into a structured sequence of actions or subgoals and decide the best path to achieve the desired outcome.
- Step 1: Generating multiple candidate plans
- Step 2: Evaluate each plan
  - Efficiency (Which is faster?)
  - Tool Availability (Which tools are available)
  - Cost (Does it require premium tools?)
  - Risk (Will it fail if we get no applicants?)
  - Alignment with constraints (remote-only? budget?)
- Step 3: Select the best plan with the help of:
  - Human-in-the-loop input (e.g., "Which of these options do you prefer?")
  - A pre-programmed policy (e.g., "Favor low-cost channels first")

What is Agentic AI

- **Reasoning:**
- Reasoning is the cognitive process through which an agentic ai system interprets information, draws conclusions, and makes decisions — both while planning ahead and while executing actions in real time.
- Reasoning During Planning:
  - Goal decomposition - Break down abstract goals into concrete steps
  - Tool selection - Decide which tools will be needed for which steps
  - Resource estimation - Estimate time, dependencies, risks
- Reasoning During Execution:
  - Decision-making - Choosing between options (3 candidates match → schedule 2 best, reject 1)
  - HITL handling - Knowing when to pause and ask for help (Unsure about salary range)
  - Error handling - Interpreting tool/API failures and recovering

What is Agentic AI

- **Adaptability:**
- Adaptability is the agent's ability to modify its plans, strategies, or actions in response to unexpected conditions — all while staying aligned with the goal.
  - Failures (e.g., Calendar API is down).
  - External feedback (e.g., Receiving fewer applications than expected).
  - Changing goals (e.g., Shifting priority to hiring a freelancer instead).

What is Agentic AI

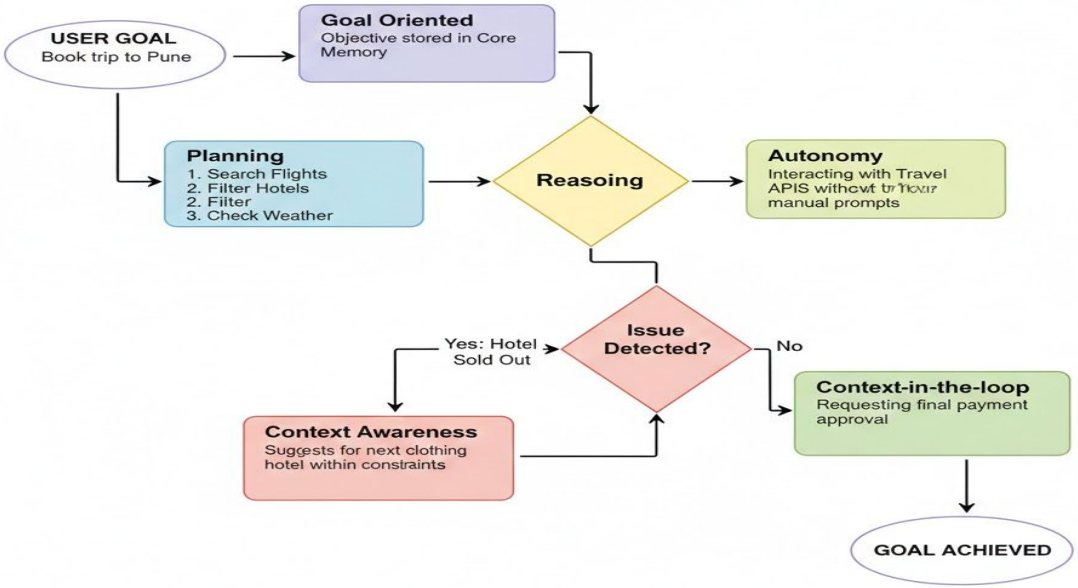
- **EXAMPLE =>**
- **Goal Oriented**: You give the agent one high-level objective: "Book a trip to Pune for a conference on March 10th." It doesn't just answer a question; it holds this goal in its "core memory" until the flight and hotel are confirmed.
- **Planning:** The agent breaks this down into steps:
  - Search for flights from your home city to Pune.
  - Filter for morning arrivals (Constraint).
  - Find a hotel within 5km of the conference venue.
- **Reasoning:** During the process, it makes logical inferences. It sees a flight that is Rs 3000 cheaper but has a 10-hour layover. It reasons that for a business trip, saving time is more important than saving Rs 3000, so it selects the direct flight.

What is Agentic AI

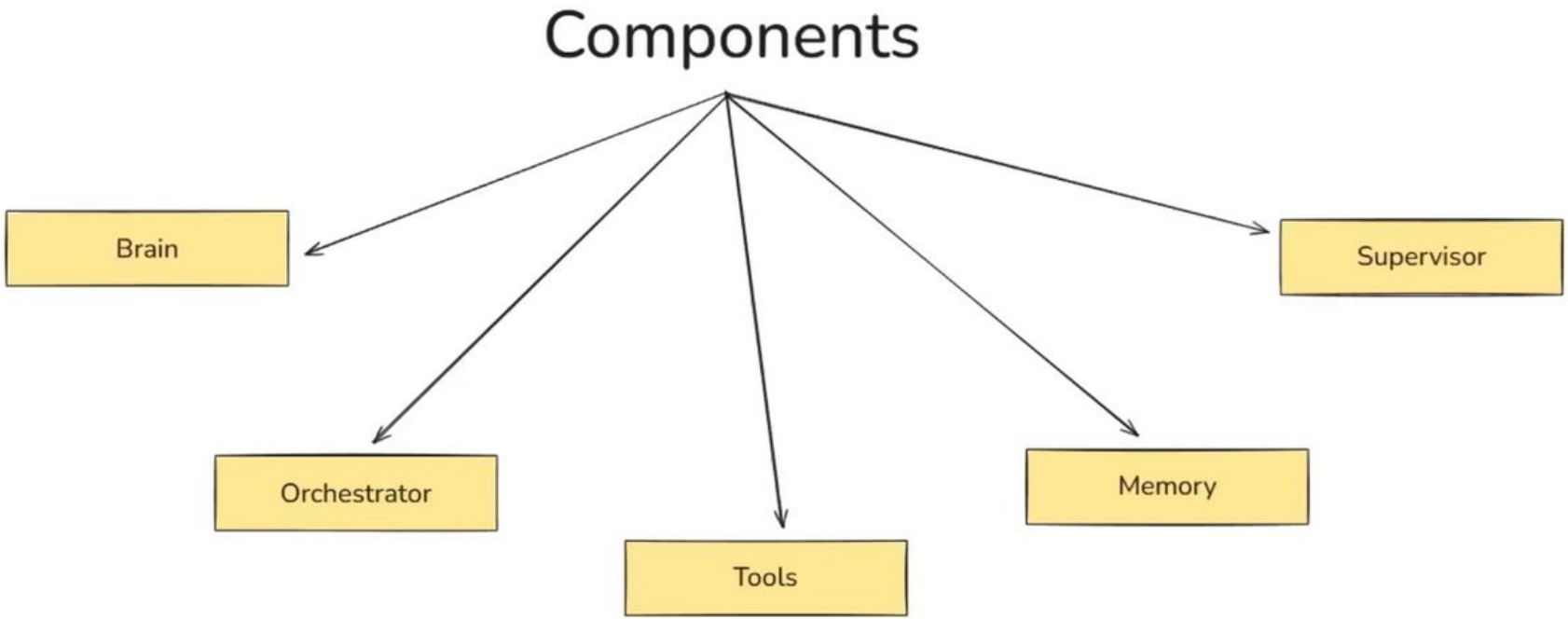
- **Autonomy:** The agent searches, compares prices, and selects the best options without asking you "Should I click search now?" for every step. However, it follows Permission Scope and pauses to ask for your final approval before charging your credit card (Human-in-the-Loop).
- **Adaptability:** While booking, the agent finds that the chosen hotel just sold out. Instead of crashing or giving up, it adapts by immediately searching for the next best hotel in the same area that fits your original constraints.
- **Context Awareness:** It knows you are traveling to Pune in February/March, so it checks the local weather and adds a note to your itinerary suggesting you pack light clothing because it will be warm.

What is Agentic AI

The Agentic Workflow: Booking a Trip to
AI Travel Assistant to Pune



Components of Agentic AI Application



Components of Agentic AI Application

Brain

Orchestrator

Goal Interpretation	Understands user instructions and translates them into objectives.
Planning	Breaks down high-level goals into subgoals and ordered steps.
Reasoning	Makes decisions, resolves ambiguity, and evaluates trade-offs.
Tool Selection	Chooses which tool(s) to use at a given step.
Communication	Generates natural language outputs for humans or other agents.
Task Sequencing	Determines the order of actions (step 1 → step 2 → ...).
Conditional Routing	Directs flow based on context (e.g., if failure, retry or escalate).
Retry Logic	Handles failed tool calls or reasoning attempts with backoff.
Looping & Iteration	Repeats steps (e.g., keep checking job apps until 10 are received
Delegation	Decides whether to hand off work to tools, LLM, or human.

Components of Agentic AI Application

Tools

External Actions	Perform API calls (e.g., post a job, send an email, trigger onboarding).
Knowledge Base Access	Retrieve factual or domain-specific information using RAG or search tools to ground responses.

Memory

Short-Term Memory	Maintains the active session's context — recent user messages, tool calls, and immediate decisions.
Long-Term Memory	Persists high-level goals, past interactions, user preferences, and decisions across sessions.
State Tracking	Monitors progress: what's completed, what's pending (e.g., "JD posted", "Offer sent").

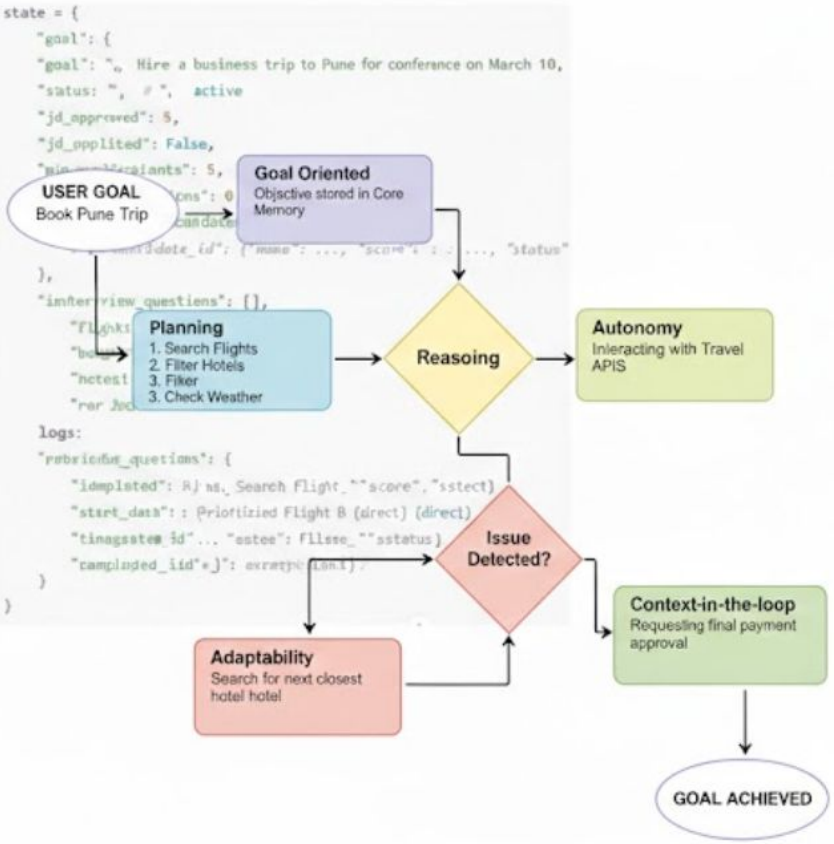
Approval Requests (HITL)	Agent checks with human before high-risk actions (e.g., sending offers).
Guardrails Enforcement	Blocks unsafe or non-compliant behavior.
Edge Case Escalation	Alerts humans when uncertainty/conflict arises.

Lang-graph

- In langgraph u represent your entire workflow as a graph
- So each task becomes a node in that graph
- And u connect these nodes with edges which controls the flow
- You can add loops or branching on the edges as well (like conditions)

Lang-graph

- **State -> Shared Memory**
- Sometimes some nodes may be very important for the entire workflow or they affect many things in the workflow because the value of these data points will evolve over time. And these data points we call as state.
- In langgraph when you are creating the graph at that time u also create the State (created with the help of pydantic or typeddict). And every node of your graph can access and read and edit this state (dict).
- So as u progress in your workflow each node will read the state and edit or update the values if necessary.



Lang-graph

- LangGraph is an orchestration framework that enables you to build stateful, multi-step, and event-driven workflows using large language models (LLMs). It’s ideal for designing both single-agent and multi-agent agentic AI applications.
- Think of LangGraph as a flowchart engine for LLMs — you define the steps (nodes), how they’re connected (edges), and the logic that governs the transitions. LangGraph takes care of state management, conditional branching, looping, pausing/resuming, and fault recovery — features essential for building robust, production-grade AI systems.

Lang-graph

- **When to Use What?**
- Use LangChain when you're building simple, linear workflows — like a prompt chain, summarizer, or a basic retrieval system.
- Use LangGraph when your use case involves complex, non-linear workflows that need:
  - Conditional paths
  - Loops
  - Human-in-the-loop steps
  - Multi-agent coordination
  - Asynchronous or event-driven execution

Lang-graph

- **Should We Still Use LangChain?**
- Yes. LangGraph is built on top of LangChain — it doesn't replace it.
- You'll still use LangChain components like:
  - ChatOpenAI (LLMs)
  - PromptTemplate
  - Retrievers
  - DocumentLoaders
  - Tools, etc.
- LangGraph handles workflow orchestration, while LangChain provides the building blocks for each step in that workflow.

Lang-graph: Graphs, Nodes and Edges

- In langgraph behind the scene each node is python function
- So basically in the graph multiple python functions are connected with each other with the help of edges.
- Edges tell us.. After executing the current node which node will get executed next. Edges can be sequential, parallel or conditional or loop

Lang-graph: Graphs, Nodes and Edges

- In LangGraph State is a shared memory that flows through your workflow. It holds all the data being passed between the nodes as your graph runs.

At the time of execution of a node we give this state to it as well and considering what node is doing it can also change or update the state. State is shared between all the nodes, it is mutable.

In code -> this state is a special dictionary called as typed dictionary (this is a class in python and u create an object of this class and add all the fields)

```
essay_text: str
topic: str
depth_score: int
language_score: int
clarity_score: int
total_score: int
feedback: Annotated[list[str], add]
evaluation_round: int
```

Lang-graph: Reducers

- Reducers in langgraph define how updates from the nodes are applied to the shared state.
- Each key in the state can have its own reducer, which determines if new data replaces, adds or merges to the existing value
- Reducer determines that in the state a node should add, merge or replace the value.
- When we are going from stage to stage in graph we call it as superstep. Here in a stage multiple steps can be there.